

Database Management Systems Lab Mini Project Report

Project Title: Student Performance Analyzer (SPA)

Technology Stack: Python (Flask), MySQL, HTML/CSS/JavaScript

1. Project Overview

The Student Performance Analyzer is a role-based web application developed to simplify and digitize college academic management. The system supports three user roles : Admin, Faculty, and Student each with secure and customized access. Admins manage users, student records, reports, and audit logs. Faculty members handle attendance, marks, assignments, and timetables, while students can view their academic details such as marks, attendance, assignments, and schedules. The system also provides real-time performance analysis and improves efficiency, accuracy, and communication within the institution.

2. Development Approach

- **Role-Based Architecture:** The system supports Admin, Faculty, and Student roles with role-specific permissions enforced in both the backend and frontend.
- **Session-Based Authentication:** Flask sessions with bcrypt password hashing provide secure authentication, while a forced password change improves account security.
- **Forgot Password System:** Admins can reset passwords through security questions, while faculty and students submit reset requests to the admin.
- **Audit Logging:** All important actions such as login, updates, attendance, and deletions are recorded in an audit log for tracking and accountability.
- **Subject Marks Management:** Marks are stored with internal, external, and total scores using an upsert query pattern for efficient insert and update operations.
- **Database View for Rankings:** A MySQL view with SQL window functions automatically calculates student ranks and percentiles, simplifying backend logic.

3. Challenges And Solutions

Maintaining Secure Role-Based Access One of the major challenges was ensuring that Admins, Faculty, and Students could only access features meant for them. Managing permissions across multiple pages and APIs became difficult as the project grew. This was solved by implementing centralized role checks in the backend along with dynamic frontend menus that automatically change based on the logged-in user's role.

Preventing Duplicate Usernames While creating new accounts, admins sometimes entered usernames that already existed, leading to confusing errors after form submission. To improve the user experience, a real-time username availability check was added with debounce functionality, allowing the system to instantly show whether a username was available or already taken.

Handling Chart Re-rendering Errors The dashboard uses Chart.js for analytics and performance visualization. During navigation between pages, charts were sometimes rendered multiple times on the same canvas, causing errors and memory issues. This problem was solved by creating a helper function that destroys existing chart instances before rendering new ones.

Dynamic Dashboard Rendering Each user role required a completely different dashboard layout and navigation menu. Managing multiple interfaces without repeating large amounts of code became challenging. To solve this, the sidebar and dashboard components were rendered dynamically based on the logged-in user's role, making the application cleaner and easier to maintain.

Managing Forgot Password Without Email Support Since the project did not include email integration, implementing password recovery required an alternative approach. Admin accounts were given a security question-based reset option, while faculty and students could send password reset requests directly to the admin for manual approval and reset.

Handling Real-Time Analytics Data Displaying attendance statistics, ranking information, and student performance analytics in real time required efficient database queries and smooth frontend updates. Optimized APIs and dynamic chart rendering techniques were used to ensure fast loading and accurate visualization of data.

4. Features of Student Performance Analyzer

Authentication & Security

- Secure login with role-based access (Admin, Faculty, Student)
- bcrypt password hashing and session-based authentication
- Forced password change on first login
- Forgot password and password reset system
- Real-time username availability checking
- Secure logout and session validation

User Management

- Create, update, enable/disable, and delete users
- Faculty and student account management
- Password reset request handling
- View all users with status and details

Student Management

- Add, edit, delete, and view student records
- Department-wise filtering and advanced search
- Student profile with marks and grades
- CSV export and student comparison charts

Marks & Academic Management

- Internal, external, and total marks management
- Semester-wise subject records
- Automatic grade calculation
- Department-wise rankings and bonus marks system
- Efficient marks update using upsert queries

Attendance Management

- Mark attendance individually or in bulk
- Attendance percentage and status tracking
- Student attendance summaries and reports
- Progress bars and attendance analytics

Timetable & Assignments

- Weekly timetable management by department
- Assignment posting, submission, and tracking
- Overdue assignment highlighting
- Faculty access to student submissions

Analytics & Reports

- Dashboard with performance statistics
- Charts and graphs using Chart.js
- Department averages and top performers
- At-risk student identification
- Leaderboards with ranks and percentiles

Ranking System

- Overall and department-wise student ranking
- Percentile calculation using SQL window functions
- Medal indicators for top ranks

Report Card Generation

- Downloadable student report cards
- Subject-wise marks and grade summaries
- Print-friendly report format

Notifications & Alerts

- Notification bell with warnings and alerts
- Low attendance and low marks notifications
- Role-based notification visibility

Audit Logging

- Tracks login, updates, attendance, marks, and deletions
- Complete audit trail with timestamps and actions

User Interface Features

- Single Page Application (SPA) design
- Dynamic dashboards and sidebars based on role
- Responsive layout with charts, modals, and toast notifications
- Real-time search, filtering, and interactive UI components

Advanced DBMS Concepts in SPA Project

- | | |
|---------------------------|--|
| • Window Functions | RANK() in student_rank_view |
| • PARTITION BY | Department wise rank in student_rank_view |
| • PERCENT_RANK() | Percentile in leaderboard and reportcard |
| • Views | student_rank_view |
| • Composite Unique Key | subject_marks, attendance, submissions |
| • UPSERT | Marks, attendance, assignment submissions |
| • Correlated Subquery | Assignment submission status check |
| • Conditional Aggregation | Attendance present count |
| • NULLIF() | Division by zero in attendance |
| • COALESCE() | NULL to zero for marks and attendance |
| • ON DELETE CASCADE | Auto cleanup on user or student delete |
| • JOIN in UPDATE | Bonus marks filtered by department |
| • LEAST() | Capping bonus marks at 100 |
| • Dynamic Query Building | Partial update of student details |
| • Parameterized Queries | SQL injection prevention everywhere |
| • CASE in ORDER BY | Timetable day of week sorting |
| • TIME_FORMAT() | Timetable time display |
| • HAVING Clause | Low marks and low attendance notifications |
| • Multi-table | JOIN Student listing with marks and username |
| • ENUM Data Type | Role and attendance status columns |